Java Backend Architecture

Interview Series

Chapter 18.2

Reverse Proxy Configuration

50+ Interview Q&A	Code Examples	Architecture Diagrams	Advanced Topics

Chapter 18.2 – Reverse Proxy Configuration

Overview

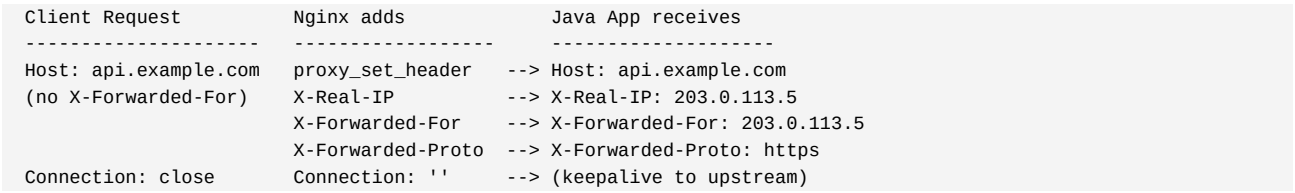
Nginx as a reverse proxy sits between clients and backend Java services, forwarding HTTP/HTTPS requests, masking backend topology, and enriching requests with headers. The core directive proxy_pass forwards traffic to an upstream server or pool. Proper header forwarding (Host, X-Forwarded-For, X-Real-IP) is essential so Java apps receive correct client context for logging, security, and personalisation. Timeout tuning and keepalive pools directly impact Java microservice throughput and resilience.

Key Definitions

Term	Definition
proxy_pass	Forwards the request to a backend or upstream pool. Supports http:// and https:// schemes.
upstream block	Defines a named pool of backend servers with load-balancing algorithm and optional keepalive pool.
proxy_set_header	Adds or overwrites a request header sent to upstream. Used to pass client IP and protocol info.
proxy_hide_header	Removes a response header from upstream before forwarding to client.
proxy_connect_timeout	Maximum time to establish TCP connection to upstream (default 60s; use 5-10s in prod).
proxy_read_timeout	Time between two successive reads from upstream. Not the total request duration.
proxy_buffering	When on (default), Nginx buffers full upstream response before sending to client.
proxy_request_buffering	When off, streams request body directly to upstream — needed for large uploads.
proxy_http_version	Set to 1.1 to enable HTTP/1.1 keepalive to upstream (required with keepalive directive).
keepalive N	Caches N idle connections per worker to upstream. Requires proxy_http_version 1.1.

Diagram 1: Reverse Proxy Request Flow

Diagram 2: Header Enrichment Flow



Code Examples

Minimal Reverse Proxy Config:

```
http {
    upstream java_api {
        server app1:8080;
        server app2:8080;
        keepalive 32;
    }
    server {
        listen 80;
        server_name api.example.com;
        location /api/ {
            proxy_pass         http://java_api/;
            proxy_http_version 1.1;
            proxy_set_header   Connection        '';
            proxy_set_header   Host              $host;
            proxy_set_header   X-Real-IP         $remote_addr;
            proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
            proxy_set_header   X-Forwarded-Proto $scheme;
            proxy_connect_timeout  5s;
            proxy_read_timeout     60s;
            proxy_next_upstream error timeout http_502 http_503;
        }
    }
}
```

Streaming Upload (no request buffering):

```
location /upload {
    proxy_pass              http://java_api;
    proxy_request_buffering off;
    proxy_buffering         off;
    proxy_read_timeout      300s;
    client_max_body_size    500m;
}
```

Security Headers on Response:

```
location / {
    proxy_pass http://java_api;
    proxy_hide_header X-Powered-By;
    add_header X-Content-Type-Options  nosniff;
    add_header X-Frame-Options         SAMEORIGIN;
    add_header X-XSS-Protection        "1; mode=block";
}
```

Interview Q&A – Reverse Proxy Configuration

Q1. What does proxy_pass http://backend/ vs http://backend do?

With trailing slash: Nginx strips the matched location prefix. location /api/ { proxy_pass http://backend/; } -- /api/users becomes /users on backend. Without trailing slash: full URI forwarded unchanged. /api/users stays /api/users. Wrong choice causes 404s in Spring controllers expecting specific path prefixes.

Q2. Why must you set proxy_http_version 1.1 with the keepalive directive?

HTTP/1.0 has no persistent connections. HTTP/1.1 is required for keepalive. Also set proxy_set_header Connection '' to clear any Connection: close the client sent, preventing Nginx from closing the upstream connection. Without both, the keepalive pool is ignored and a new TCP connection is made per request.

Q3. How does $proxy_add_x_forwarded_for differ from $remote_addr?

$remote_addr is the immediate TCP connection IP (could be a load balancer). $proxy_add_x_forwarded_for appends $remote_addr to any existing X-Forwarded-For header, building a chain: client_ip, lb_ip. Java apps using Spring's ForwardedHeaderFilter read this chain to get the real client IP for logging and rate limiting.

Q4. How does proxy_read_timeout work?

proxy_read_timeout is the time between two successive read operations from upstream, not total request duration. If backend sends nothing for this duration, Nginx returns 504. Set per-location: 60s for REST APIs, 300-3600s for SSE/long-polling, 300s+ for batch processing. Pair with application-level timeouts shorter than Nginx timeouts.

Q5. Explain proxy_next_upstream safety implications.

proxy_next_upstream retries on the next server for: error, timeout, http_502, http_503. Safe for idempotent methods (GET, HEAD). Dangerous for POST -- retrying a payment causes duplicates. Restrict with proxy_next_upstream_methods GET HEAD. Limit retries with proxy_next_upstream_tries 2 and proxy_next_upstream_timeout.

Q6. What is proxy_intercept_errors?

When on, Nginx intercepts upstream 4xx/5xx responses and serves custom error pages defined by error_page, instead of passing raw upstream errors to clients. Useful for consistent error pages across Java microservices. Without this, Spring Boot error JSON bypasses Nginx error handling.

Q7. What headers should every Java reverse proxy set?

Host: $host (virtual host routing in Spring). X-Real-IP: $remote_addr. X-Forwarded-For: $proxy_add_x_forwarded_for. X-Forwarded-Proto: $scheme (HTTPS detection for Spring's server.forward-headers-strategy). X-Request-ID: $request_id for distributed tracing correlation across Nginx and Java logs.

Q8. How does proxy_buffering off affect streaming endpoints?

With proxy_buffering off, each chunk from upstream is immediately forwarded to the client. Required for SSE (Spring WebFlux Flux), chunked responses, large file streaming. Trade-off: upstream connections held longer (tied to slow client speed). Apply per-location to avoid global performance impact.

Q9. What is proxy_ssl_verify and when is it needed?

When proxy_pass uses https://, proxy_ssl_verify on enables certificate verification (set proxy_ssl_trusted_certificate to CA bundle). Prevents man-in-the-middle between Nginx and Java backend. proxy_ssl_certificate/key for mutual TLS to backend. proxy_ssl_session_reuse on reuses TLS sessions in keepalive pools.

Q10. How do you implement CORS at Nginx for Java backends?

Use map to whitelist origins: map $http_origin $cors { ~^https://(app|admin).example.com$ $http_origin; } add_header Access-Control-Allow-Origin $cors always. Handle preflight: if ($request_method = OPTIONS) { add_header Access-Control-Allow-Methods 'GET POST PUT'; return 204; } Centralises CORS at gateway, not duplicated in each Java service.

Q11. Explain proxy_cache_bypass.

proxy_cache_bypass $http_authorization -- if the variable is non-empty, bypass cache and fetch from upstream. Pair with proxy_no_cache to prevent storing. Essential for authenticated requests: prevents serving cached private data from one user to another. X-Cache-Status header (add_header X-Cache-Status $upstream_cache_status) shows HIT/MISS.

Q12. How do you pass client certificate info to Java backend with mTLS?

After ssl_verify_client on, Nginx exposes $ssl_client_s_dn (Distinguished Name), $ssl_client_verify (SUCCESS/FAILED), $ssl_client_escaped_cert (URL-encoded cert). Pass via proxy_set_header X-Client-DN $ssl_client_s_dn. Java backend reads header for audit logging and authorisation without re-validating the certificate.